

23. Arrays of structures

Herman Kamper

The previous lecture introduced structures: a way to define your own type by bundling related variables together under one name. A single `struct coord` or a single `struct student` is fine, but structures earn their keep when you need many of them at once: a whole class of students, a whole deck of cards, a whole periodic table of elements.

That combination, an array whose elements are structures, is what this lecture is about. If you understand structures and you understand arrays, this is mostly just putting the two things next to each other. There is a bigger example at the end that pulls together everything from the last few lectures, bringing together structures, arrays and pointers.

Arrays of structures

Structures are very often used to store a record of something: one student's name and grade, one element's chemical properties. And you very rarely want just one record. You want a list of them.

Take the periodic table. Each element has a name, an atomic number and an atomic mass. One element is naturally a structure:

```
struct element
{
    char *name;
    int atomicNumber;
    double atomicMass;
};

typedef struct element Element;
```

A single element is easy enough:

```
Element someElement;
someElement.name = "Lithium";
someElement.atomicNumber = 3;
someElement.atomicMass = 6.94;
```

But there are 118 elements. What we actually want is an array, exactly the way you would declare an array of ints or doubles, except that the type of each element is `Element`:

```
Element periodicTable[118];
```

That line creates 118 boxes, each big enough to hold a whole Element. You reach a particular one with ordinary subscript notation, and once you have picked out one element of the array, you reach into it with the dot, exactly as you would for any other structure variable:

```
periodicTable[0].name = "Hydrogen";  
periodicTable[0].atomicNumber = 1;  
periodicTable[0].atomicMass = 1.00794;  
  
printf("%s has an atomic number %d and a mass of %f.\n",  
       periodicTable[0].name, periodicTable[0].atomicNumber,  
       periodicTable[0].atomicMass);
```

This prints:

Hydrogen has an atomic number 1 and a mass of 1.007940.

Read `periodicTable[0].atomicNumber` from left to right: `periodicTable[0]` picks out one box from the array, a whole Element, and `.atomicNumber` then picks out one member of that particular box.

What this looks like in memory

`Element periodicTable[118];` creates 118 boxes side by side, and each of those boxes is itself made up of three smaller boxes, because that is what an Element is: an 8-byte pointer box for name, a 4-byte box for the atomicNumber integer, and an 8-byte box for atomicMass decimal number. The array does not know or care that its elements happen to be structures; it just reserves 118 blocks in which we can put Elements.

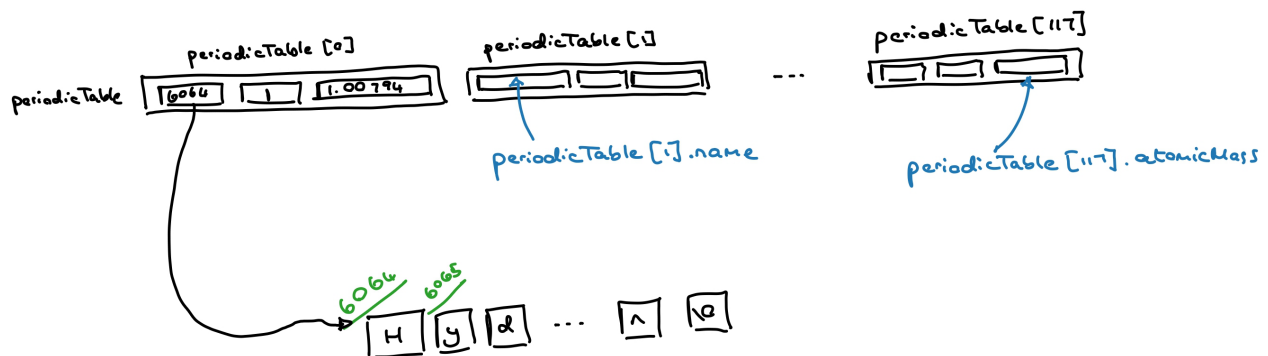


Figure 1: `periodicTable` in memory, after `periodicTable[0]` has been filled in. `periodicTable` on its own is the address of the first box. Each `periodicTable[i]` is itself three boxes: a pointer, an int and a double. The string "Hydrogen" lives elsewhere in memory as its own run of character boxes, and `periodicTable[0].name` just holds the address 6064 where that run starts.

Two things are worth being precise about here. First, `periodicTable[1].name` is a name for a box in exactly the same way that `periodicTable[0].name` is: first the array picks out element 1, then the dot picks out its name member.

Second, look at what happens when you write a string literal into a `char *` member:

```
periodicTable[0].name = "Hydrogen";
```

This is the same thing that happens whenever you assign a string literal to a `char *`. Somewhere in memory, C lays out the individual characters H, y, d, r, o, g, e, n, and then a `\0` to mark the end, one byte each (because they are `char`), sitting next to each other. Say that run starts at address 6064. What goes into the name box is not the string itself, just that one address, 6064. The structure holds a pointer to where the string starts, not the string.

Structures, pointers and arrays together

You already know that you can point at a structure, and you already know that you can point at an element of an array. Let us just make sure these concepts are in place by looking at a line like:

```
Element *elementPtr = &periodicTable[0];
```

`elementPtr` holds the address of `periodicTable[0]`, the address where this structure starts. From here you have three equivalent ways to reach a member through the pointer, and this is a good moment to make sure you have all three:

```
printf("%s has an atomic number %d and a mass of %f.\n",
      (*elementPtr).name, elementPtr->atomicNumber,
      elementPtr[0].atomicMass);
```

`(*elementPtr).name` dereferences the pointer to get the structure, then takes its name member; the parentheses are required, for the same reason they always are, dereferencing has to happen before the dot. `elementPtr->atomicNumber` is the arrow shorthand for exactly that. And `elementPtr[0].atomicMass` treats the pointer as if it were an array: `[0]` means “dereference, with an offset of zero”, which is again the same operation. The result is:

Hydrogen has an atomic number 1 and a mass of 1.007940.

In practice you will see `elementPtr->member` far more often than the other two. It reads the most naturally, “follow the pointer and take this member”, and it is what you will use almost every time a function receives a pointer to a structure, which is most of the time once you start writing functions that operate on arrays of structures.

Dynamic arrays of structures

Everything above declared the array's size up front, 118, baked into the program. Exactly as with arrays of `int` or `double`, you can instead reserve the array at runtime with `malloc`, and nothing about the pattern changes just because the elements are structures this time:

```
int numberOfElements = 118;
Element *periodicTable;

periodicTable = malloc(numberOfElements * sizeof(Element));

if (periodicTable == NULL)
{
    return 1;
}

periodicTable[0].name = "Hydrogen";

free(periodicTable);
```

`malloc` is asked for `numberOfElements * sizeof(Element)` bytes, exactly enough room for that many whole structures back to back. What comes back is the address of the first one, stored in a plain `Element *` (called `periodicTable`). From that point on, `periodicTable` behaves exactly like the fixed-size array from before: subscript it, dot into a member, pass it to functions. The pointer does not know or care whether the memory it is pointing at came from a declaration or from `malloc`; only the array's owner, you, has to remember that this one needs a matching `free` when you are done with it.

A bigger example: representing a deck of cards

The example that pulls the lecture together. In a previous lecture, we developed a program to represent a deck of cards. A card has two properties that both need representing at once: its face (ace, two, three, and so on up to king) and its suit (hearts, diamonds, clubs, spades). Earlier in the course, before structures existed as an option, the natural way to hold both at once was a two-dimensional array, four rows for the suits and thirteen columns for the faces, with each cell holding a number saying where that card sits in the shuffled deck. You can review that lecture [here](#).

Our strategy worked, but it meant remembering, every time you touched the array, that row 2 column 11 was not “the number 47”, it was “the queen of clubs”, and you had to carry that mapping around in your head. The question now is: given that you know about structures, could you represent the whole deck with a one-dimensional array instead?

Start with the structure. A card is a face and a suit, both naturally represented as strings:

```
struct card
{
    const char *face;
    const char *suit;
};

typedef struct card Card;
```

face and suit are ordinary pointers here. Pointers and arrays of characters are close enough to interchangeable that a `char *` is the natural way to hold a name like "King" or "Hearts". The `const` says these two members will only ever point at fixed strings like "King", never at something the program intends to edit.

Now the deck is a single array of 52 of these Cards:

```
Card deck[52];
```

That one line replaces the entire two-dimensional table from the previous lecture. `deck[0]` through `deck[51]` are 52 boxes, and each box already knows its own face and suit; there is no row and column to translate back into a card, because each element of the array *is* a card. The structure carries the meaning that used to live in where a number sat in a 2D grid.

You could fill it in card by card, writing the strings out directly:

```
deck[0].face = "Ace";
deck[0].suit = "Hearts";
deck[1].face = "Deuce";
deck[1].suit = "Hearts";
```

but 52 lines of that gets old fast, and it is easy to typo a suit name differently in two places. A cleaner approach, and the one used in practice, is to define the thirteen faces and four suits once, as their own arrays of strings, and then have each card point into them:

```
const char *faces[] = {"Ace", "Deuce", "Three", "Four", "Five", "Six", "Seven",
                       "Eight", "Nine", "Ten", "Jack", "Queen", "King"};
const char *suits[] = {"Hearts", "Diamonds", "Clubs", "Spades"};

deck[0].face = faces[0];
deck[0].suit = suits[0];
deck[1].face = faces[1];
deck[1].suit = suits[0];
```

Read `deck[0].face = faces[0]`; carefully, because there is a pointer being copied here, not a string. `faces[0]` is itself a `char *`, the address of the "Ace" string sitting somewhere in memory. Copying it into `deck[0].face` does not duplicate the string; it just makes `deck[0].face` point at the exact same characters that `faces[0]` already points at. Every ace in the deck, whatever its suit, can point at that one shared "Ace" string. The picture below shows the whole arrangement:

the deck array in the middle, and the suit and face arrays on either side that the cards' pointers reach into.

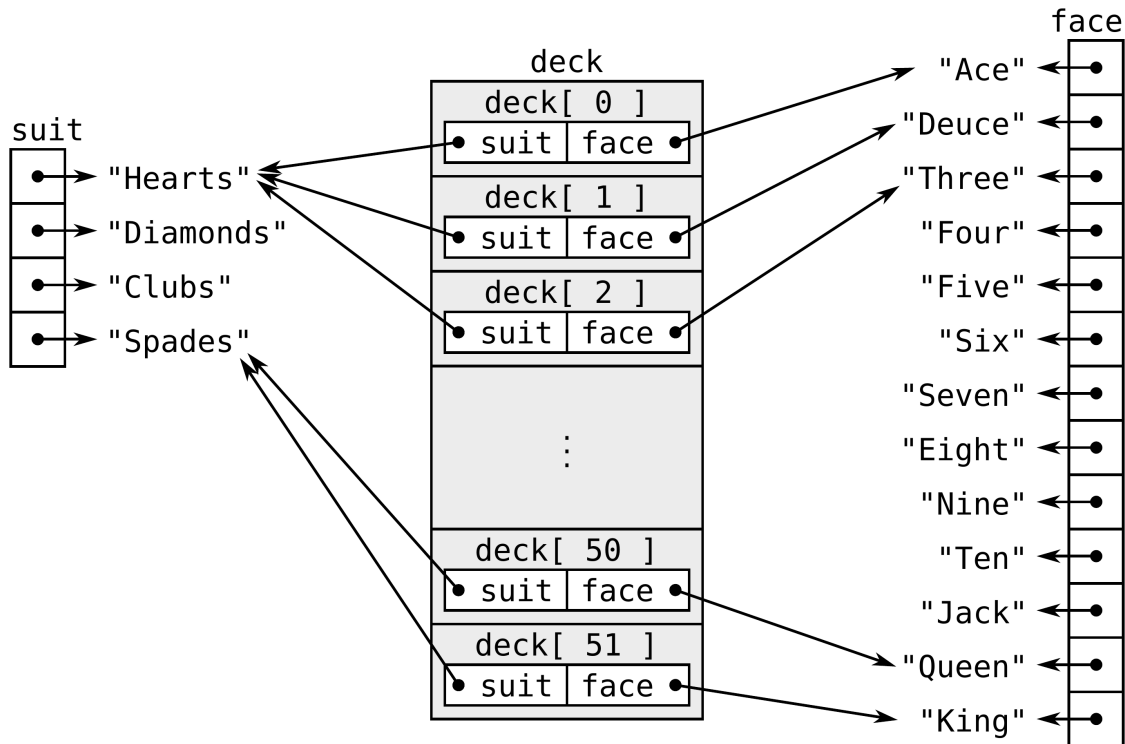


Figure 2: The complete data structure. deck is an array of 52 Card structures. Each card's suit and face members are pointers that reach out to one of the four strings in suit or one of the thirteen strings in face; the strings themselves are never copied, only pointed at.

Once the deck is filled, you can print a card the same way regardless of how its pointers got set:

```
printf("First card is %s of %s\n", deck[0].face, deck[0].suit);
```

which, with the assignments above, prints:

```
First card is Ace of Hearts
```

Filling, shuffling and dealing the whole deck

Putting all of this together gives a complete card program. The program splits the work into three functions, each one a small loop over the 52-element array. It is worth trying to write this yourself before looking at the answer.

```
/*
 * Card shuffling and dealing, using a one-dimensional array of structures.
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct card
{
    const char *face;
    const char *suit;
};

typedef struct card Card;

void fillDeck(Card * const wDeck, const char * wFace[], const char * wSuit[]);
void shuffle(Card * const wDeck);
void deal(const Card * const wDeck);

int main(void)
{
    Card deck[52];

    const char *face[] = {"Ace", "Deuce", "Three", "Four", "Five", "Six", "Seven",
                          "Eight", "Nine", "Ten", "Jack", "Queen", "King"};
    const char *suit[] = {"Hearts", "Diamonds", "Clubs", "Spades"};

    srand(time(NULL));

    fillDeck(deck, face, suit);
    shuffle(deck);
    deal(deck);

    return 0;
}

// place strings into Card structures
void fillDeck(Card * const wDeck, const char * wFace[], const char * wSuit[])
{

```

```

    int i;

    for (i = 0; i <= 51; i++)
    {
        wDeck[i].face = wFace[i % 13]; // remainder gives the face, 0 to 12
        wDeck[i].suit = wSuit[i / 13]; // quotient gives the suit, 0 to 3
    }
}

// shuffle cards
void shuffle(Card * const wDeck)
{
    int i;
    int j;
    Card temp;

    for (i = 0; i <= 51; i++)
    {
        j = rand() % 52;
        temp = wDeck[i];
        wDeck[i] = wDeck[j];
        wDeck[j] = temp;
    }
}

// deal cards
void deal(const Card * const wDeck)
{
    int i;

    for (i = 0; i <= 51; i++)
    {
        printf("%5s of %-8s", wDeck[i].face, wDeck[i].suit);
        if ((i + 1) % 4)
            printf(" ");
        else
            printf("\n");
    }
}

```

A few things are worth pointing out.

fillDeck builds an ordered deck in one pass, without ever writing “Ace” or “Hearts” by hand. As *i* runs from 0 to 51, *i* % 13 cycles through 0 to 12 four times over, which is exactly the face index, and *i* / 13 steps from 0 to 3 every thirteen cards, which is the suit index. So cards 0 to 12 are all hearts, ace through king, then 13 to 25 are all diamonds, and so on. Compare this with

the two-dimensional version, where the suit and face were the row and column you indexed with directly; here they are recovered from a single running index using % and /.

`shuffle` uses a different idea from the 2D version. Rather than searching for an empty slot to drop a random position into, it walks through the deck once and, at each position `i`, picks a random index `j` anywhere in the deck and swaps whatever is sitting at `i` with whatever is sitting at `j`. Notice the line `temp = wDeck[i];`; this copies an entire `Card` structure, both its face and suit pointers, in one assignment, exactly as you saw in the previous lecture for a single structure variable. Swapping two structures is three of these whole-structure copies, no different in kind from swapping two ints.

`deal` just walks the shuffled deck and prints it, four cards to a line. The function parameters `const Card * const wDeck` are worth a second look if they seem unfamiliar: the first `const` says the function will not modify the cards being pointed at, and the second says it will not modify the pointer `wDeck` itself either, a stricter promise than you have usually needed, but a reasonable one for a function whose whole job is to read and print.

Running the program gives something like:

```
    Six of Diamonds    Ace of Clubs        Six of Clubs        Nine of Diamonds
Seven of Hearts      Jack of Diamonds Three of Diamonds Deuce of Hearts
Three of Hearts      Five of Clubs        Deuce of Spades     Ace of Diamonds
    Ace of Spades    Deuce of Clubs        Queen of Spades     King of Clubs
```

Every run gives a different order, because `srand(time(NULL))` seeds the random number generator from the clock; the deck is always a complete, valid deck, just shuffled differently each time.

Summary

Arrays of structures are the pattern behind almost every list of real-world things you will ever store in a program: students, elements, cards, employees, sensor readings. The recipe is always the same: Decide what one record needs to hold, and write that down as a `struct`. Decide how many records you need, fixed or worked out while the program runs, and declare an array or `malloc` one. Everything else (subscripting, the dot, the arrow, passing the array to functions), is exactly what you already know from arrays and from structures separately.

Exercises

1. Write the `Element/periodicTable` example from the start of this lecture as a complete program, add a second element at index 1 (helium: atomic number 2, atomic mass 4.0026), and print both using a for loop instead of writing out two separate `printf` calls.
2. Rewrite the fixed `Element periodicTable[118];` declaration to use `malloc` instead, including a `NULL` check, fill in hydrogen as before, print it back, and free the array.
3. Using `fillDeck` as a guide, write a function `int countSuit(const Card *deck, const char *suitName)` that counts how many cards in a filled, unshuffled 52-card deck have a given suit. Confirm it returns 13 for every suit. *Hint:* look at the `strcmp` function.
4. `shuffle` swaps `wDeck[i]` with `wDeck[j]` for a random `j`, including the case where `j` happens to equal `i`. Explain why that case is harmless, and why, no matter what sequence of swaps happens, the deck afterwards is guaranteed to still contain exactly 52 distinct cards, none lost and none duplicated.
5. Write a function `Card highestCard(const Card *deck, int n)` that returns a copy of whichever of the first `n` cards in an ordered (unshuffled) deck has the highest face, where face order runs Ace low, King high. You may find it easiest to compare the position of each card's face inside the `face[]` array from `main` rather than comparing strings directly.
6. Define a struct `student` with a name and three test marks in an array member (`double marks[3]`), declare an array of 5 students, fill in some marks, and write a function `double average(struct student s)` that returns one student's average. Note that this passes the whole structure, marks array included, by value: say what that costs compared to passing a pointer, for a structure this size.